



**UNIVERSIDAD TECNOLÓGICA
DE PANAMÁ FACULTAD DE
INGENIERÍA DE SISTEMAS
COMPUTACIONALES
DEPARTAMENTO DE
COMPUTACIÓN Y
SIMULACIÓN DE SISTEMAS**

**CARRERA LICENCIATURA EN INGENIERÍA DE SISTEMA Y
COMPUTACIÓN HERRAMIENTA DE COMPUTACIÓN GRÁFICA**

LABORATORIO #5

**Título del Taller: Fundamentos de
animación 2D**

Integrantes:

Rodríguez, Lysmarie 8-1010-386

Moreno, Dereck 8-1004-1420

Campine, Kevin 8-1028-1050

Aparicio, Cristhofer 9-765-102

Chauhan, Gulam 8-1043-2255

Profesor:

Ing. Samuel Jiménez

SEMESTRE I, 2026

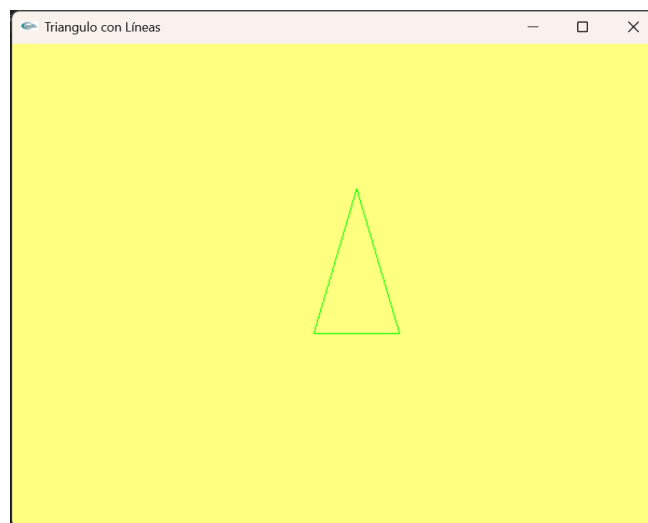
INTRODUCCIÓN

En el presente laboratorio se desarrollaron actividades relacionadas con el uso de primitivas básicas en OpenGL, con el propósito de comprender cómo se construyen figuras gráficas a partir de instrucciones y coordenadas. Durante la experiencia se trabajó con diferentes comandos para dibujar líneas, triángulos, cuadrados, polígonos y círculos, aplicando colores, grosores y modificaciones en la proyección de la ventana. Además, se analizaron distintas primitivas como GL_LINES, GL_TRIANGLES, GL_LINE_LOOP, GL_LINE_STRIP, GL_QUADS y GL_POLYGON, observando cómo cada una influye en la forma final representada en pantalla. Estas prácticas permitieron fortalecer el manejo básico de OpenGL y aplicar los conocimientos adquiridos en el desarrollo de diseños gráficos sencillos, como cuadros, triángulos, figuras con líneas y una casa utilizando primitivas básicas.

1. En el void proyección realice las siguientes modificaciones:

```
void proyeccion()  
{  
    glMatrixMode(GL_PROJECTION);  
    gluOrtho2D(-15, 15, -5, 5);  
    glClearColor(1.0, 1.0, 0.5, 0.5);  
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen

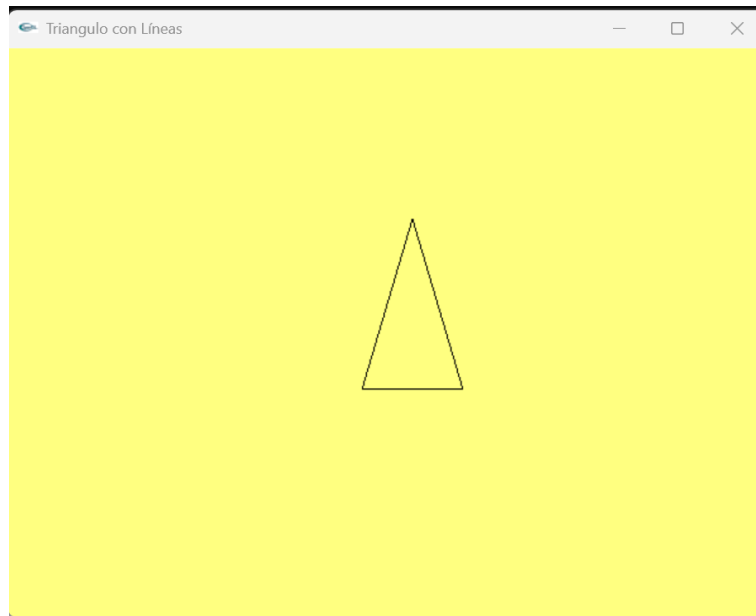


Al ejecutar, la ventana muestra un fondo de color amarillo, el triángulo disminuye. Las coordenadas irán de -15 a 15 en el eje X, y de -5 a 5 en el eje Y.

1. Realice la siguiente modificación a la función void triangulo.

```
void Triangulo()  
{  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(0.0, 0.0, 0.0);  
    glBegin(GL_LINES);  
    glVertex2i(1.5, 2);  
    glVertex2i(-1, -1);  
    glVertex2i(-1, -1);  
    glVertex2i(3, -1);  
    glVertex2i(3, -1);  
    glVertex2i(1.5, 2);  
    glEnd();  
    glFlush();  
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



El triángulo cambia de un color verde a un negro.

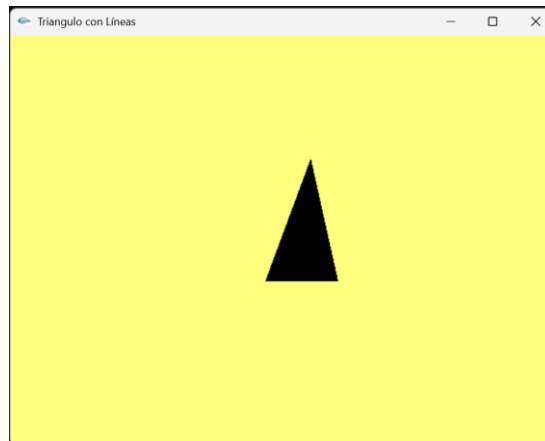
2. Manteniéndonos en la función Triangulo, realicemos la siguiente modificación.

```
void Triangulo()
{
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0,0.0,0.0);

    glBegin(GL_TRIANGLES);
        glVertex2f(1.5, 2.0);
        glVertex2f(-1.0, -1.0);
        glVertex2f(3.0, -1.0);
    glEnd();
    glFlush();
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



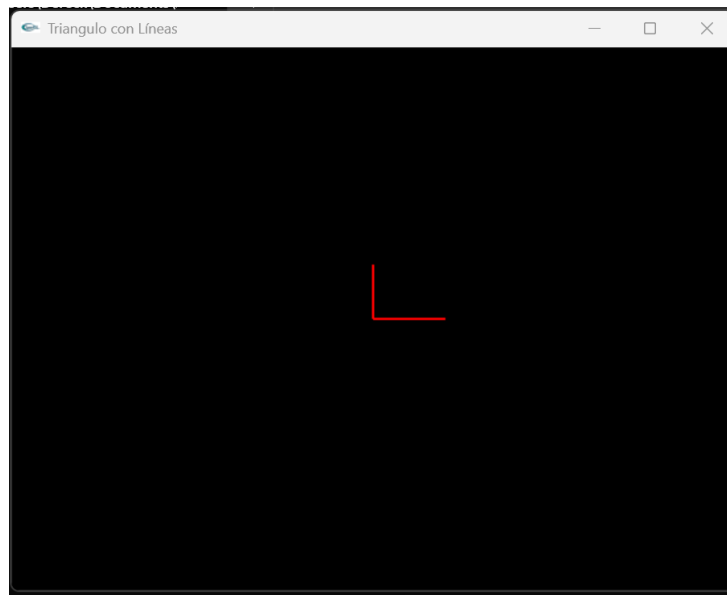
El cambio de GL_LINES a GL_TRIANGLES hace que agruparse de tres en tres para formar triángulos rellenos.

- Tanto en el void proyección como en el void triangulo aplicamos las siguientes modificaciones.

```
void proyeccion()
{
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-5,5,-5,5);
    glClearColor(0.0,0.0,0.0,0.0);
}
```

```
void Triangulo()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,0.0,0.0);
    glEnable(GL_LINE_SMOOTH);
    glBegin(GL_LINES);
        glVertex3f(0.0f,0.0f,0.0f);
        glVertex3f(1.0f,0.0f,0.0f);
        glVertex3f(0.0f,0.0f,0.0f);
        glVertex3f(0.0f,1.0f,0.0f);
        glVertex3f(0.0f,0.0f,0.0f);
        glVertex3f(0.0f,0.0f,1.0f);
    glEnd();
    glFlush();
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



El color del triángulo cambia a rojo y ahora solo se ve una L, tiene una configuración para un plano cartesiano en 3ra dimensión, pero al ser configurado en 2D solo vemos la L ya que el eje Z apunta hacia nosotros.



4. Con base en el punto 4, comente o elimine la instrucción `glEnable(GL_LINE_SMOOTH)`. ¿Qué ocurre al hacerlo? Explique el resultado observado.

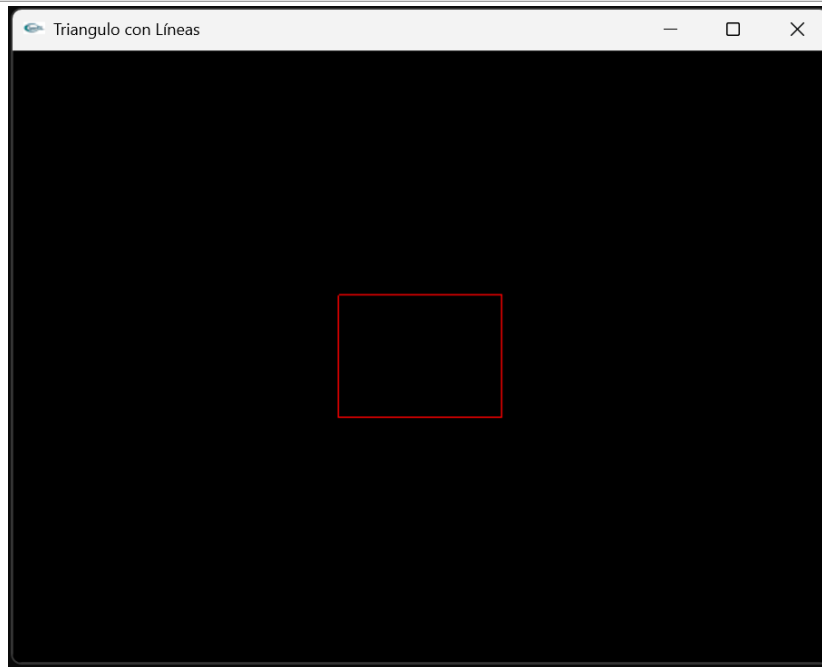


Se ve la L menos resaltada, ya que pierde la calidad visual que aporta el suavizado de esta función y pasa a tener un renderizado de líneas puro y geométrico con bordes duros y pixelados.

5. A continuación, utilice la siguiente instrucción dentro de la función Triangulo:

```
void Triangulo()  
{  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0,0.0,0.0);  
  
    glBegin(GL_LINE_LOOP);  
        glVertex2f(-1.0f, -1.0f);  
        glVertex2f( 1.0f, -1.0f);  
        glVertex2f( 1.0f,  1.0f);  
        glVertex2f(-1.0f,  1.0f);  
    glEnd();  
  
    glFlush();  
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



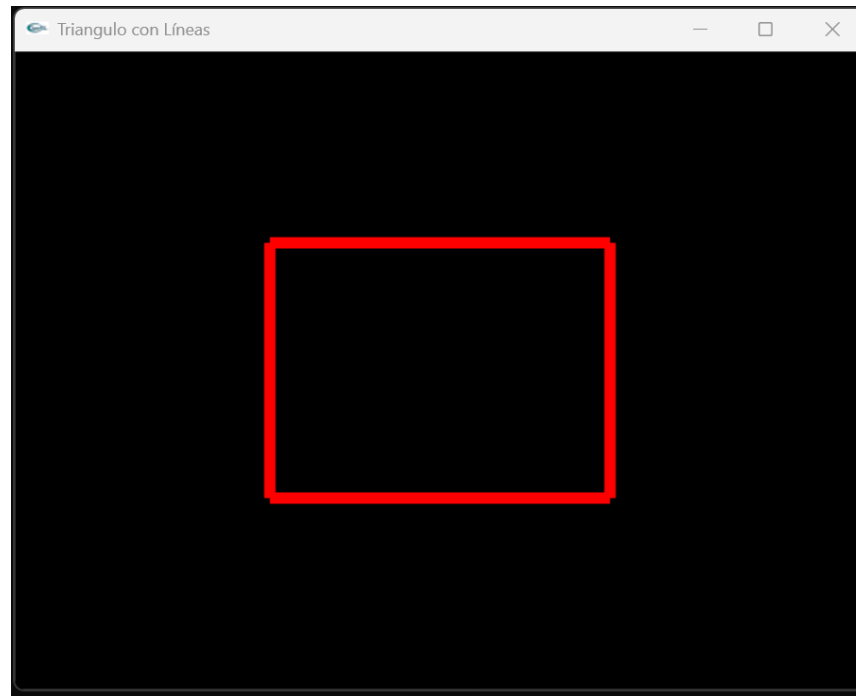
Ahora la figura cambió a un cuadrado rojo vacío. La ventaja clave de `GL_LINE_LOOP` es que, de forma automática, traza una última línea desde el último vértice hasta el primero. Además, definimos los límites claro del cuadrado.

6. En la función void triangulo realizamos los siguientes cambios.

```
void Triangulo()  
{  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0,0.0,0.0);  
    glLineWidth(8.0);  
    glBegin(GL_LINE_LOOP);  
        glVertex2f(-2.0f, -2.0f);  
        glVertex2f( 2.0f, -2.0f);  
        glVertex2f( 2.0f,  2.0f);  
        glVertex2f(-2.0f,  2.0f);  
    glEnd();  
    glFlush();  
}
```




Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen

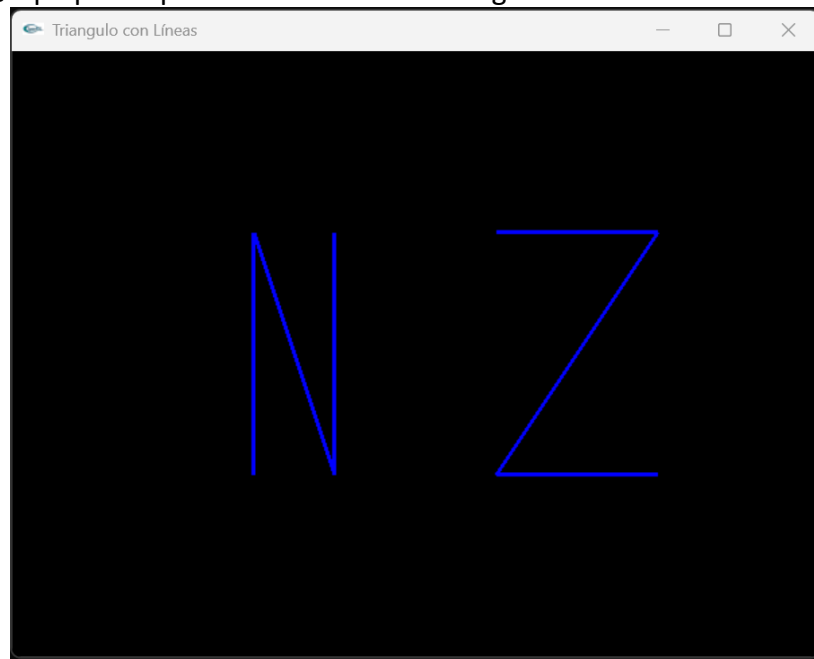


Ahora el cuadrado se ve con las líneas más gruesas, ya que cambiamos el grosor de 1.0pixel a 8.0 con `glLineWidth(8.0)`; y al cambiar las coordenadas el cuadrado abarca el doble de espacio.

7. Seguimos realizando modificaciones en void triangulo, esta vez con este comando:

```
void Triangulo(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0,0.0,1.0);
    glLineWidth(3.0);
    glBegin(GL_LINE_STRIP);
        glVertex2f(-2, -2);
        glVertex2f(-2, 2);
        glVertex2f(-1, -2);
        glVertex2f(-1, 2);
    glEnd();
    glBegin(GL_LINE_STRIP);
        glVertex2f(1, 2);
        glVertex2f(3, 2);
        glVertex2f(1, -2);
        glVertex2f(3, -2);
    glEnd();
    glFlush();
}
```

Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



Cambiamos el color de rojo a azul, con menos espesor (3.0píxeles) y ahora nos muestra una N y una Z, ya que GL_LINE_STRIP conecta los vértices en secuencia con líneas continuas, pero no une el último vértice con el primero. Deja la figura abierta y las letras es debido a las coordenadas que le dimos.

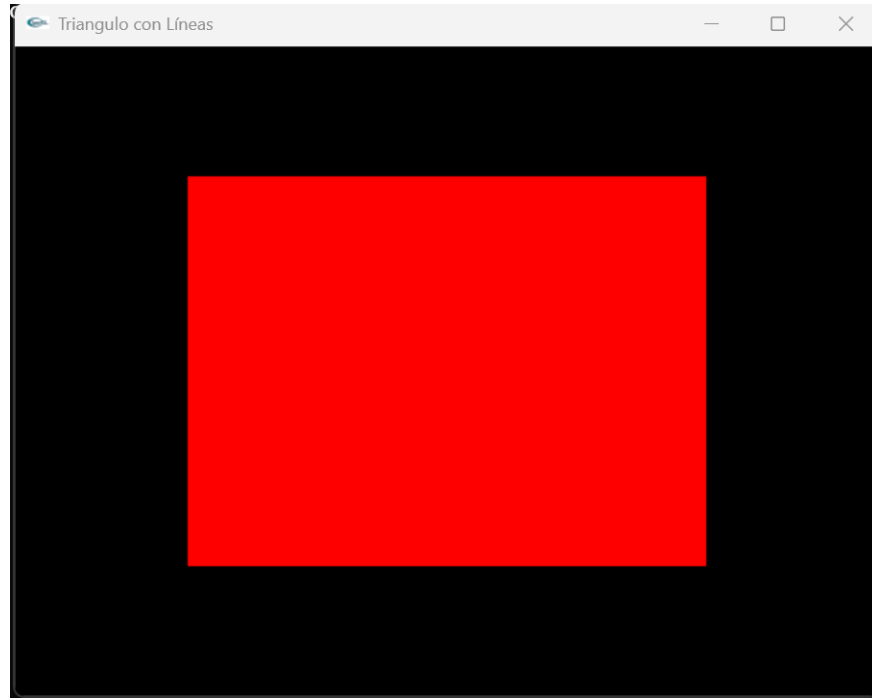


8. Aplicamos la última modificación a void triangulo, establece este comando:

```
void Triangulo() {  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1.0,0.0,0.0);  
    glBegin(GL_QUADS);  
        glVertex2f(-3.0f, -3.0f);  
        glVertex2f( 3.0f, -3.0f);  
        glVertex2f( 3.0f,  3.0f);  
        glVertex2f(-3.0f,  3.0f);  
    glEnd();  
    glFlush();  
}
```



Ejecute el programa, ¿Explique lo que sucede? Plasme la imagen



Ahora vemos un cuadrado relleno de color rojo, ya que GL_QUADS toma los 4 vértices que definiste y los utiliza para construir un cuadrilátero completamente relleno de color sólido, y con sus nuevas dimensiones por las coordenadas dadas, este cuadrado es más grande.

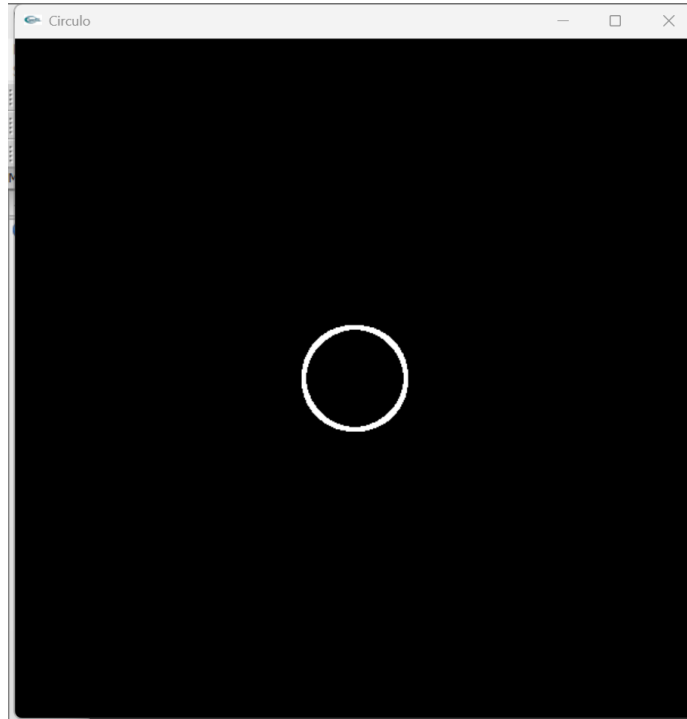
9. Mencione las diferencias y ventajas entre GL_LINES, GL_TRIANGLES, GL_LINE_LOOP, GL_POINTS, GL_POLYGON, GL_LINE_STRIP, GL_QUADS

Primitivas	Diferencias	Ventajas
GL_LINES	Dibuja segmentos de línea independientes entre pares de vértices.	Permite crear múltiples líneas sin conexión entre ellas. Ideal para ejes o trazos simples.
GL_TRIANGLES	Cada grupo de 3 vértices forma un triángulo independiente.	Muy eficiente para representar superficies y polígonos complejos. Base de la mayoría de modelos 3D.
GL_LINE_LOOP	Conecta los vértices en orden y une el último con el primero, formando un lazo.	Útil para dibujar contornos cerrados como cuadrados, polígonos o marcos.
GL_POINTS	Cada vértice se dibuja como un punto en pantalla.	Ideal para representar nubes de puntos, píxeles o figuras como círculos mediante coordenadas.
GL_POLYGON	Conecta todos los vértices en orden para formar un polígono relleno	Permite dibujar figuras cerradas de cualquier número de lados.
GL_LINE_STRIP	Conecta los vértices en orden, formando una línea continua sin cerrar.	Útil para gráficos de curvas, trayectorias o funciones matemáticas.
GL_QUADS	Cada grupo de 4 vértices forma un cuadrilátero independiente.	Muy práctico para dibujar rectángulos, cuadros y superficies planas.



10. Ahora proceda a ejecutar el programa denominado laboratorio 2.2

¿Explique lo que sucede?, Plasme la imagen



Al compilar y correr este programa, se abre una ventana con fondo negro, y en todo el centro un círculo perfecto delineado por puntos blancos con un radio de 3 unidades dentro del plano cartesiano de tamaño 20.

11. Realice las modificaciones en void proyección y void display del laboratorio 2.2

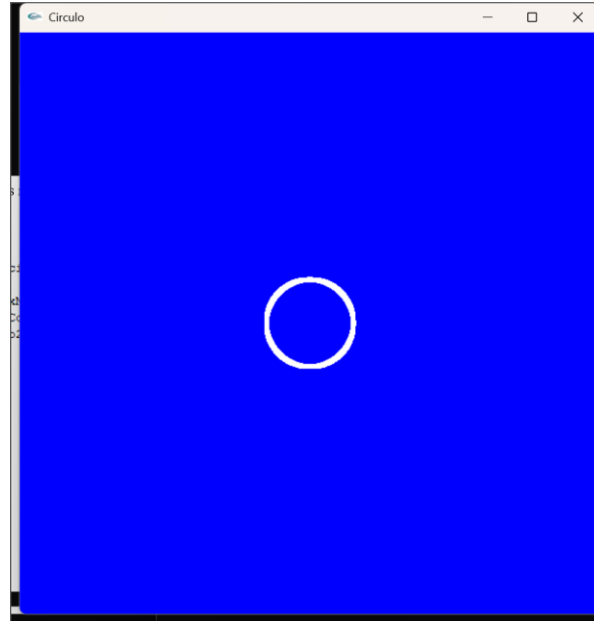
```
void proyeccion(void)
{
    glMatrixMode(GL_PROJECTION);
    glClearColor(0.0,0.0,1.0,1.0);
    gluOrtho2D(-20,20,-20,20);
}
```

```
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,1.0,1.0);
    glPointSize(5.0);
    glBegin(GL_POINTS);

    for (double i=0.0; i<10; i+=0.001)
    {
        calx=radio*cos(i);
        caly=radio*sin(i);
        glVertex2f(calx,caly);
    }
    glEnd();
    glFlush();
}
```

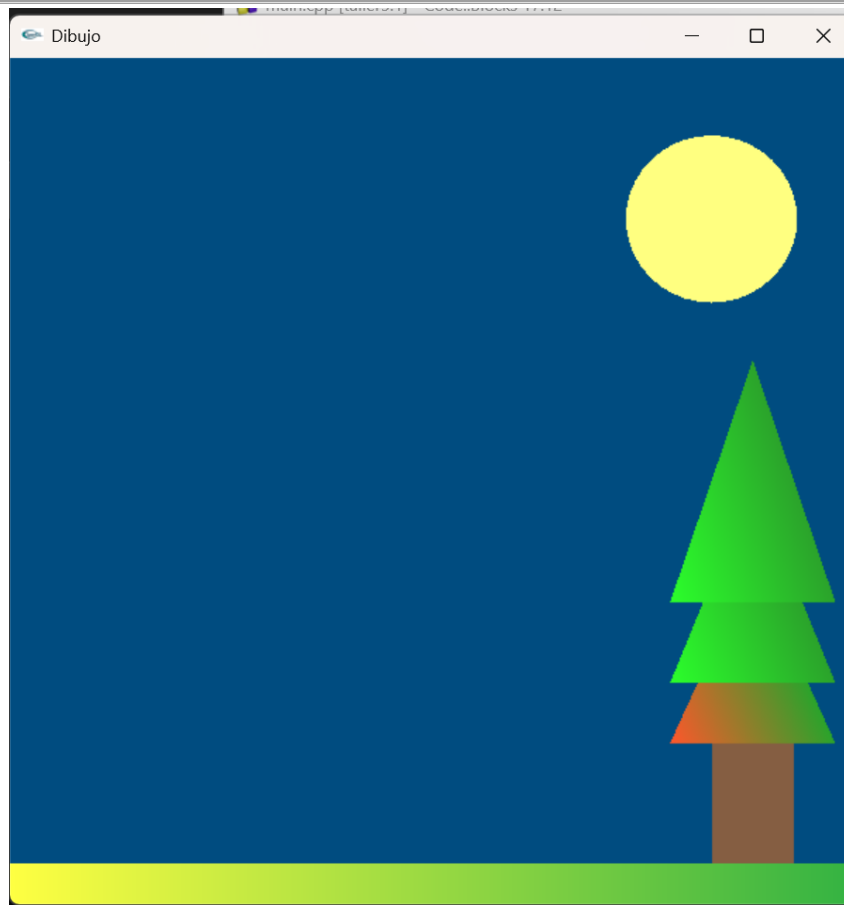


¿Explique lo que sucede?, Plasme la imagen



Se cambia el fondo a azul, también se hace un poco más gruesos los puntos y el cambio del ciclo for hace que la circunferencia se dibuje completa y 60% en la segunda vuelta

12. Ejecute ahora el laboratorio 2.3, ejecute, que es lo que se visualiza en pantalla. Plasme su imagen

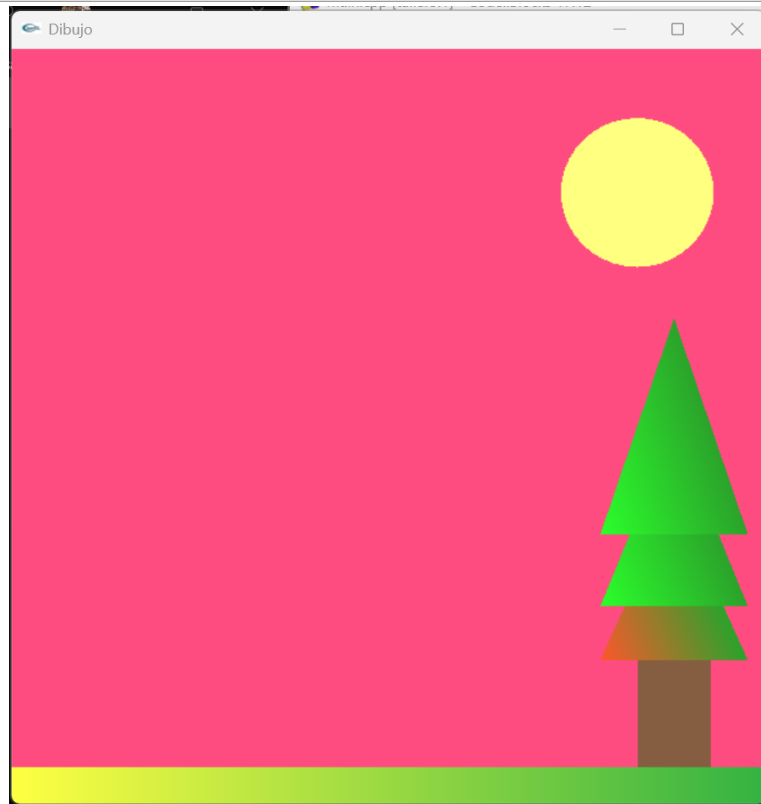


En este código se compone un paisaje completo en el lado derecho de la pantalla (un árbol de copas triangulares, un suelo y una luna usando una esfera sólida).

13. Apliquemos la siguiente modificación al void iniciarproyeccion

```
void iniciarProyeccion()
{
    glClearColor(1.0,0.3,0.5,0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-20,21,-22,20);
}
```

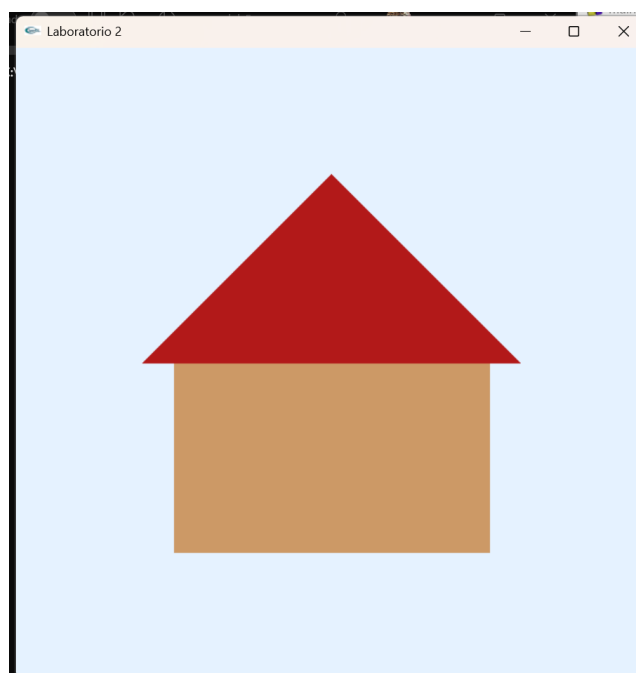
Ejecute, que es lo que se visualiza en pantalla. Plasme su imagen



Cambiamos la combinación de colores del fondo del paisaje agregándole 1.0 de rojo

14. Ejecute el laboratorio 2.4, que es lo que se visualiza en pantalla. Plasme su imagen

Se ve una ventana de 600 x 600 píxeles con un fondo azul claro muy limpio, y en el centro una casa clásica: paredes color café claro/beige con un tejado triangular rojo bien proporcionado.





UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS
LABORATORIO 5

FC-FISC-1-8-2016



15. En el laboratorio anterior #4 aprendiste a dibujar líneas en OpenGL utilizando la función `glLineWidth` para controlar su grosor.

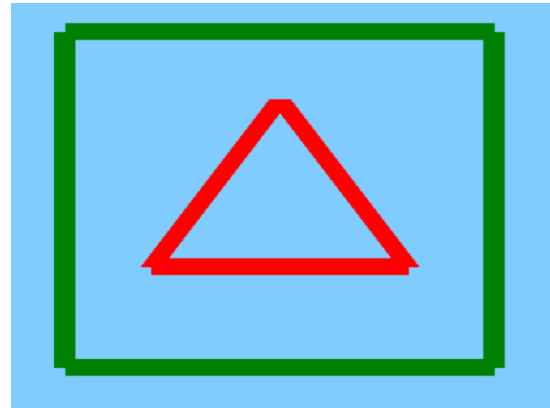
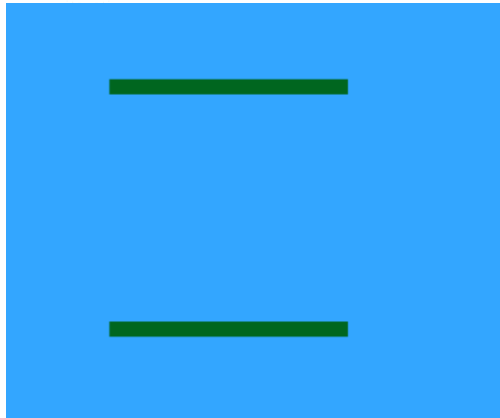
Con base en ese conocimiento, desarrolle un programa en OpenGL que cumpla con los siguientes requisitos:

1. Dibuje un cuadro utilizando líneas, manteniendo el mismo grosor trabajado previamente.

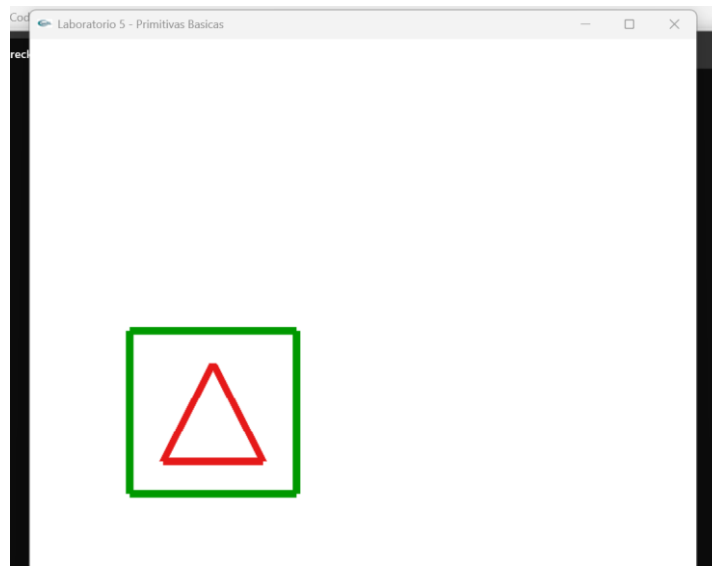


UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS
LABORATORIO 5

FC-FISC-1-8-2016



Dentro del cuadro, dibuje un triángulo, también utilizando líneas del mismo grosor. Ambas figuras deben estar correctamente posicionadas, de manera que el triángulo quede completamente dentro del cuadro. Utilice colores distintos para diferenciar el cuadro y el triángulo.



```
#include <windows.h>

#include <GL/glut.h>


// Función para configurar el área de proyección y el fondo de la pantalla
void iniciarProyeccion(void)
{
    // Configura la matriz de proyección
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Define los límites del espacio bidimensional (-10 a 10 en ambos ejes)
    gluOrtho2D(-10.0, 10.0, -10.0, 10.0);

    // Color de fondo de la ventana (Blanco Puro)
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
}


// Función principal de renderizado
void display(void)
{
    // Limpia la pantalla aplicando el color de fondo (Blanco)
    glClear(GL_COLOR_BUFFER_BIT);

    // Controlar el grosor general de las líneas a 8.0 como pide el laboratorio
    glLineWidth(8.0f);
```

```
// RETO 16: CUADRO CON TRIÁNGULO INTERNO (Lado Izquierdo)
```

```
// 1. Dibujo del Cuadrado (Verde)
```

```
glColor3f(0.0f, 0.6f, 0.0f);
```

```
glBegin(GL_LINE_LOOP);
```

```
    glVertex2f(-7.0f, -4.0f); // Inferior izquierdo
```

```
    glVertex2f(-2.0f, -4.0f); // Inferior derecho
```

```
    glVertex2f(-2.0f, 1.0f); // Superior derecho
```

```
    glVertex2f(-7.0f, 1.0f); // Superior izquierdo
```

```
glEnd();
```

```
// 2. Dibujo del Triángulo Interno (Rojo)
```

```
glColor3f(0.9f, 0.1f, 0.1f);
```

```
glBegin(GL_LINE_LOOP);
```

```
    glVertex2f(-4.5f, 0.0f); // Cúspide
```

```
    glVertex2f(-6.0f, -3.0f); // Esquina inferior izquierda
```

```
    glVertex2f(-3.0f, -3.0f); // Esquina inferior derecha
```

```
glEnd();
```

```
// Fuerza el renderizado de los dibujos en el buffer
```

```
glFlush();
```

```
}
```

```
// Función principal del programa
```

```
int main(int argc, char** argv)
```

```
{
```

```
// Inicializa la librería GLUT

glutInit(&argc, argv);


// Configura el modo de despliegue (Búfer único y modo de color RGB)

glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);


// Define el tamaño de la ventana (cuadrada de 700x700 píxeles)

glutInitWindowSize(700, 700);


// Define la posición inicial de la ventana en el escritorio

glutInitWindowPosition(100, 100);


// Crea la ventana con el título correspondiente

glutCreateWindow("Laboratorio 5 - Primitivas Basicas");


// Llama a la configuración de coordenadas y fondo

iniciarProyeccion();


// Registra la función encargada de dibujar

glutDisplayFunc(display);


// Entra en el ciclo infinito de ejecución de GLUT

glutMainLoop();

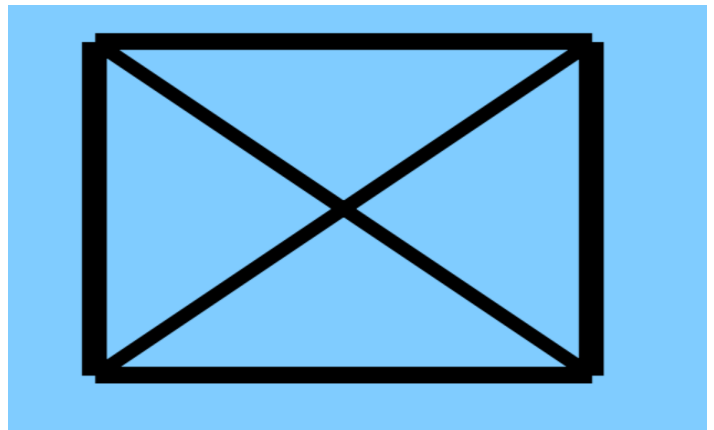

return 0;

}
```

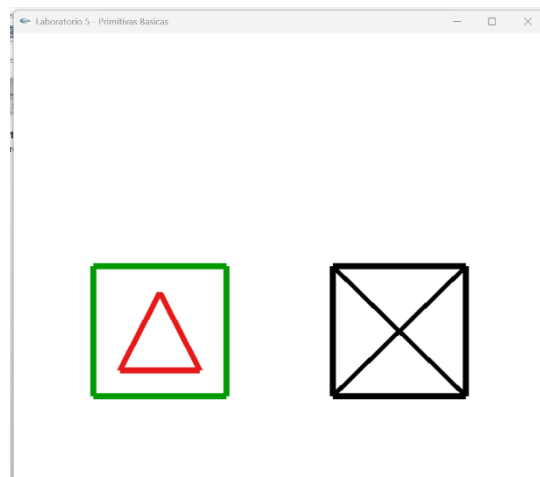
Observación: Pegue como resultado el código que su equipo de trabajo desarrolló para el resultado final.

16. Establezca el grosor de línea utilizando la función `glLineWidth`, manteniendo el mismo valor trabajado en el laboratorio anterior (#4). Dibuje un cuadro utilizando la primitiva `GL_LINE_LOOP`, asegurándose de que la figura quede correctamente cerrada. Dentro del cuadro, dibuje una "X" utilizando la primitiva `GL_LINES`, trazando dos diagonales que conecten las esquinas opuestas del cuadro. Asegúrese de que tanto el cuadro como la "X":

- Utilicen el mismo grosor de línea
- Estén alineados correctamente.
- Queden centrados en la pantalla.
- Asigne color negro a las figuras para que contrasten con el fondo.
- Compile y ejecute el programa, verificando que el resultado sea un cuadro con una "X" en su interior.



Observación: Pegue como resultado el código que su equipo de trabajo desarrolló para el resultado final.



```

#include <windows.h>
#include <GL/glut.h>

// Función para configurar el área de proyección y el fondo de la pantalla
void iniciarProyeccion(void)
{
    // Configura la matriz de proyección
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Define los límites del espacio bidimensional (-10 a 10 en ambos ejes)
    gluOrtho2D(-10.0, 10.0, -10.0, 10.0);

    // Color de fondo de la ventana (Blanco Puro)
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
}

// Función principal de renderizado
void display(void)
{
    // Limpia la pantalla aplicando el color de fondo (Blanco)
    glClear(GL_COLOR_BUFFER_BIT);

    // Controlar el grosor general de las líneas a 8.0 como pide el laboratorio
    glLineWidth(8.0f);

    // RETO 16: CUADRO CON TRIÁNGULO INTERNO (Lado Izquierdo)

    // 1. Dibujo del Cuadrado (Verde)
    glColor3f(0.0f, 0.6f, 0.0f);
    glBegin(GL_LINE_LOOP);
        glVertex2f(-7.0f, -4.0f); // Inferior izquierdo
        glVertex2f(-2.0f, -4.0f); // Inferior derecho
        glVertex2f(-2.0f, 1.0f); // Superior derecho
        glVertex2f(-7.0f, 1.0f); // Superior izquierdo
    glEnd();

    // 2. Dibujo del Triángulo Interno (Rojo)
    glColor3f(0.9f, 0.1f, 0.1f);
    glBegin(GL_LINE_LOOP);
        glVertex2f(-4.5f, 0.0f); // Cúspide
        glVertex2f(-6.0f, -3.0f); // Esquina inferior izquierda
        glVertex2f(-3.0f, -3.0f); // Esquina inferior derecha
    glEnd();

    // RETO 17: CUADRO CON UNA "X" (Lado Derecho)

    // 1. Dibujo del Cuadrado Perimetral (Negro)
    glColor3f(0.0f, 0.0f, 0.0f);
    glBegin(GL_LINE_LOOP);
        glVertex2f(2.0f, -4.0f); // Inferior izquierdo

```

```

glVertex2f(7.0f, -4.0f); // Inferior derecho
    glVertex2f(7.0f, 1.0f); // Superior derecho
    glVertex2f(2.0f, 1.0f); // Superior izquierdo
glEnd();

// 2. Dibujo de las Diagonales cruzadas (Negro)
glBegin(GL_LINES);
    // Primera diagonal: de inferior izquierdo a superior derecho
    glVertex2f(2.0f, -4.0f);
    glVertex2f(7.0f, 1.0f);

    // Segunda diagonal: de superior izquierdo a inferior derecho
    glVertex2f(2.0f, 1.0f);
    glVertex2f(7.0f, -4.0f);
glEnd();

// Fuerza el renderizado de los dibujos en el buffer
glFlush();
}

// Función principal del programa
int main(int argc, char** argv)
{
    // Inicializa la librería GLUT
    glutInit(&argc, argv);

    // Configura el modo de despliegue (Búfer único y modo de color RGB)
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);

    // Define el tamaño de la ventana (cuadrada de 700x700 píxeles)
    glutInitWindowSize(700, 700);

    // Define la posición inicial de la ventana en el escritorio
    glutInitWindowPosition(100, 100);

    // Crea la ventana con el título correspondiente
    glutCreateWindow("Laboratorio 5 - Primitivas Basicas");

    // Llama a la configuración de coordenadas y fondo
    iniciarProyeccion();

    // Registra la función encargada de dibujar
    glutDisplayFunc(display);

    // Entra en el ciclo infinito de ejecución de GLUT
    glutMainLoop();

    return 0;
}

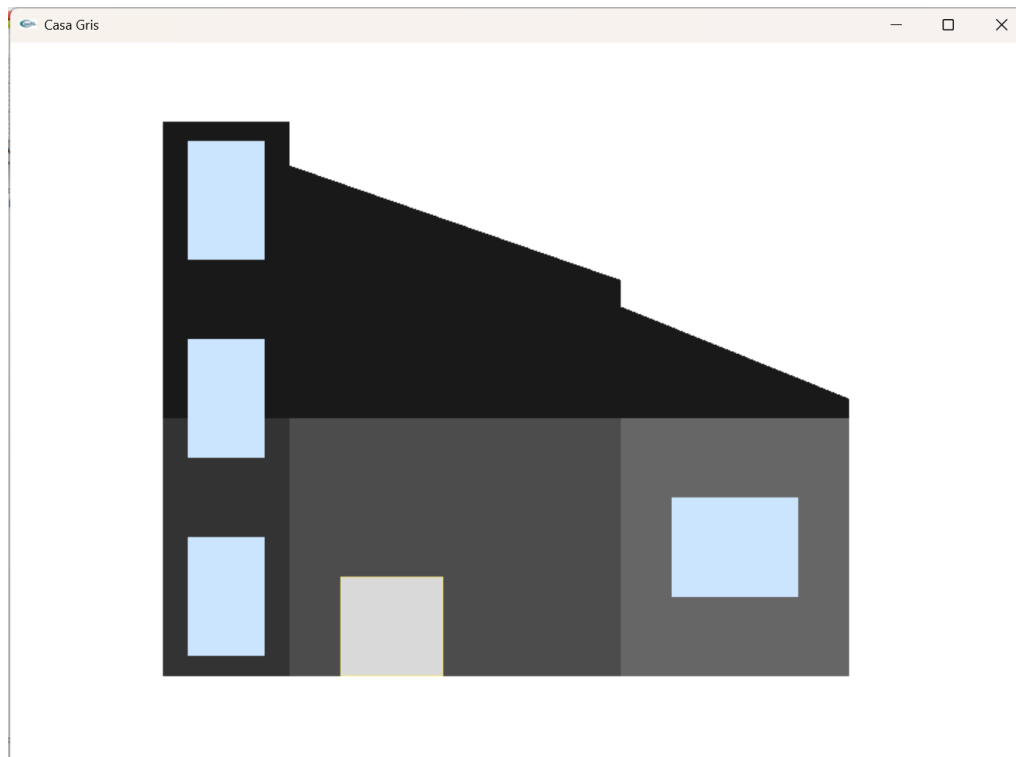
```




Reto del Laboratorio 2:

De acuerdo con lo que se aprendió en clase y en el desarrollo del laboratorio, utilizando las primitivas básicas dibujaran en OpenGL un diseño de una casa.

Ejemplo





CONCLUSIÓN

En conclusión, este laboratorio permitió reforzar los conocimientos sobre el dibujo de primitivas básicas en OpenGL y su aplicación en la creación de figuras bidimensionales. A través de la ejecución y modificación de los programas, se pudo observar cómo los cambios en las coordenadas, colores, grosor de líneas y tipos de primitivas alteran directamente la visualización de los objetos en pantalla. También se comprendió la importancia de utilizar correctamente funciones como `glLineWidth`, `glColor3f`, `gluOrtho2D` y las distintas instrucciones de dibujo para obtener resultados precisos y organizados. Finalmente, el desarrollo de los retos propuestos permitió aplicar de manera práctica los conceptos aprendidos, fortaleciendo la creatividad, el trabajo en equipo y la comprensión de los fundamentos de la computación gráfica.